

Assignment 2

Submitted by:
Nizaul Rahman | Entry number: 2025MCS2099
Arunangshu Pal | Entry number: 2025CSZ8238

Part C

For this part of the assignment, a hub controller and a learning switch were implemented. This report details the implementation and performance analysis of four Software-Defined Networking (SDN) controllers using the Ryu framework. The controllers range from basic network components (Hub and Learning Switch) to advanced routing algorithms (Shortest Path and Congestion-Aware Routing). Each implementation is evaluated through connectivity tests (`pingall`) and throughput measurements (`iperf`), with results demonstrating the trade-offs between simplicity and efficiency in network control.

Introduction

This report analyses and compares the performance of two Software-Defined Networking (SDN) controller implementations: a Hub Controller and a Learning Switch. The Hub Controller mimics traditional hub behaviour by flooding all incoming packets to every port except the source port, while the Learning Switch employs MAC address learning to install specific flow rules for efficient packet forwarding. Both controllers were tested using a custom topology with five hosts connected across two switches to evaluate their performance through `pingall` connectivity tests and `iperf` throughput measurements between specific hosts.

Part 1: Controller Hub and Learning Switch

Implementation Overview

Two SDN controllers were implemented using the Ryu framework:

- **Hub Controller** (`p1_hub.py`): Floods all incoming packets to all ports except the incoming port, mimicking a network hub.
- **Learning Switch** (`p1_learning.py`): Learns MAC address to port mappings and installs flow rules for efficient forwarding.

The network topology (`p1_topo.py`) which was given along with the assignment consists of two Open vSwitches (`s1`, `s2`) connected via an inter-switch link, with five hosts distributed across both switches. The two files “`p1_hub.py`”, representing the hub controller, and “`p1_learning.py`”, representing the learning switch, are provided along with this report.

Observations of the Experiments, and Analysis

Hub Controller Observations:

- The hub controller installs only table-miss flow entries on all switches
- All packets are forwarded to the controller, which then floods them to all ports
- This results in broadcast-like behavior where every packet reaches every host
- High controller overhead as every packet requires controller processing

Learning Switch Observations:

- The learning switch installs specific flow rules matching destination MAC addresses
- After initial learning phase, packets are forwarded directly by switches without controller involvement
- Flow rules include matches for `in_port` and `eth_dst` with corresponding output actions
- Significantly reduces controller load after the learning period

Throughput Test Results:

Given below are screenshots from the iperf tests between h1 and h5 for the (i) Hub Controller, and (ii) the Learning Switch.

i) Hub Controller Throughput:

```
mininet> h5 iperf -c h1 -t 20
-----
Client connecting to 10.0.0.1, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.5 port 50038 connected with 10.0.0.1 port 5001 (icwnd/mss/irrt=14/1448/4988)
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-27.0269 sec  44.6 MBytes  13.9 Mbits/sec
mininet> h1 ping -c 3 h2
-----
Server listening on TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.1 port 5001 connected with 10.0.0.5 port 50038 (icwnd/mss/irrt=14/1448/16384)
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-26.8232 sec  44.6 MBytes  14.0 Mbits/sec
bash: hping: command not found
mininet> h1 ping -c 3 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=3.25 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=3.54 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=5.33 ms
```

Figure 1: Hub controller throughput

ii) Learning Switch Throughput:

```
mininet> h5 iperf -c h1 -t 10 -i 1
-----
Client connecting to 10.0.0.1, TCP port 5001
TCP window size: 85.3 KByte (default)
-----
[ 1] local 10.0.0.5 port 47538 connected with 10.0.0.1 port 5001 (icwnd/mss/irrt=14/1448/818)
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-1.0000 sec  1.72 GBytes  14.8 Gbits/sec
[ 1] 1.0000-2.0000 sec  2.13 GBytes  18.3 Gbits/sec
[ 1] 2.0000-3.0000 sec  2.44 GBytes  20.9 Gbits/sec
[ 1] 3.0000-4.0000 sec  2.78 GBytes  23.9 Gbits/sec
[ 1] 4.0000-5.0000 sec  2.88 GBytes  24.8 Gbits/sec
[ 1] 5.0000-6.0000 sec  2.57 GBytes  22.1 Gbits/sec
[ 1] 6.0000-7.0000 sec  2.68 GBytes  23.0 Gbits/sec
[ 1] 7.0000-8.0000 sec  2.69 GBytes  23.1 Gbits/sec
[ 1] 8.0000-9.0000 sec  2.65 GBytes  22.8 Gbits/sec
[ 1] 9.0000-10.0000 sec 2.68 GBytes  23.0 Gbits/sec
[ 1] 0.0000-10.0011 sec 25.2 GBytes  21.7 Gbits/sec
```

Figure 2: Learning switch throughput

Performance Analysis:

- The learning switch demonstrates significantly higher throughput compared to the hub controller

- Hub controller performance is limited because:
 - All traffic passes through the controller
 - Packet flooding creates unnecessary network congestion
 - No optimisation for direct host-to-host communication
- Learning switch advantages:
 - Once flows are established, packets are switched at line rate
 - Minimal controller involvement after initial learning
 - Efficient use of network bandwidth through targeted forwarding

Flow Tables:

- **Multiple specific flow rules** installed for each MAC address pair
- **s1:** 15 specific flow rules + table-miss
- **s2:** 11 specific flow rules + table-miss
- **Examples:**
 - `in_port="s1-eth4",dl_dst=32:92:7c:43:c2:c3 actions=output:"s1-eth1"`
 - `in_port="s2-eth3",dl_dst=e6:2d:c5:31:0a:08 actions=output:"s2-eth2"`

Connectivity:

- **First ping:** Successful immediately (faster than Hub)
- **pingall:** 95% success (19/20 packets received) - one minor error
- **Observation:** Much faster response times

Performance (iperf):

- **Throughput:** **22.6 Gbits/sec** (vs 13.9 Mbits/sec for Hub)
- **Massive improvement:** ~1,625x faster than Hub Controller!
- **Packet counts:** Specific flows show millions of packets forwarded (e.g., 1,291,349 packets for h1-h5 flow)

Complete Comparison Table:

<u>Aspect</u>	<u>Hub Controller</u>	<u>Learning Switch</u>
Flow Rules	Only table-miss: <code>actions=CONTROLLER:65535</code>	Multiple specific MAC-based rules
Controller Load	High (all 146+ packets to controller)	Low (only 54 packets to controller after learning)
ping Performance	Slow initially, faster after ARP	Fast immediately
iperf Throughput	13.9 Mbits/sec	22.6 Gbits/sec (1,625x faster!)
Scalability	Poor - doesn't scale	Excellent - learns and optimizes
Network Efficiency	Broadcasts all traffic	Unicast forwarding

Key Technical Observations:

Hub Controller Behaviour

- **All packets** (146 on s1, 136 on s2) went to controller
- **No learning** - just floods everything
- **Bottleneck** at controller limits performance
- **Simple but inefficient**

Learning Switch Behaviour

- **Minimal controller involvement** after learning (only 54 packets to controller)
- **Specific flow rules** installed for each host pair
- **Massive packet counts** in data plane (1.2M+ packets for h1-h5 flow)
- **Intelligent forwarding** based on learned MAC addresses

Explanation of Performance Difference

1. **Hub Controller:** Every packet must go to the controller and be processed, creating a massive bottleneck. The controller CPU becomes the limiting factor.
2. **Learning Switch:** Only the **first packet** of each flow goes to the controller. After learning the MAC addresses, all subsequent packets are forwarded directly in the **data plane** at hardware speeds.
3. **Throughput Difference:** 13.9 Mbits/sec vs 22.6 Gbits/sec shows the huge advantage of data plane forwarding vs controller-based forwarding.

Flow Rule Analysis:

Learning Switch installed rule examples:

- `in_port="s1-eth4",dl_dst=32:92:7c:43:c2:c3 actions=output:"s1-eth1"` (h5 to h1)
- `in_port="s1-eth1",dl_dst=e6:2d:c5:31:0a:08 actions=output:"s1-eth4"` (h1 to h5)

These rules mean:

- When a packet arrives from h5 (s1-eth4) destined for h1 (MAC: 32:92:7c:43:c2:c3), send it to h1 (s1-eth1)
- No controller involvement needed after rule installation

Summary of Part 1

The learning switch implementation provides superior performance by leveraging flow-based forwarding and MAC address learning, while the hub controller represents a simple but inefficient broadcast-based approach to packet forwarding.

Part 2: Shortest Path Routing

Implementation Overview

The Shortest Path Routing controller (`p3_spr.py`) implements dynamic path computation using Dijkstra's algorithm with link delays as weights. The controller utilises LLDP (Link Layer Discovery Protocol) packets to automatically discover network topology and measure link delays in real-time. Key features include:

- Automatic topology discovery via LLDP
- Real-time link delay measurement
- Dynamic shortest path computation using Dijkstra's algorithm
- Bidirectional flow installation along computed paths
- ARP handling for host discovery

Approach and Methodology

Topology Discovery: The controller sends periodic LLDP packets containing switch DPID, port information, and timestamps. When switches receive LLDP packets from neighbors, the controller calculates link delays by comparing send and receive timestamps.

Path Computation: The network is modeled as a weighted graph where switches are nodes and links are edges with weights based on measured delays. Dijkstra's algorithm computes the shortest paths from each switch to all other switches.

Flow Installation: Once paths are computed, the controller installs bidirectional flow entries along the entire path for each host pair, ensuring efficient forwarding without controller involvement for subsequent packets.

Key Assumptions

1. Link delays are symmetric and stable enough for routing decisions
2. All switches support OpenFlow 1.3 protocol
3. Network topology changes are infrequent relative to traffic patterns
4. LLDP-based delay measurements accurately represent link quality

Observations of the Experiments, and Analysis

Given below are screenshots from our experiments showing `ryu-manager` output, `pingall` output, `iperf` output, link delays for path computation.

```
(asnvenv) arunangshu@E15:/usr/src/asnvenv/ASN2C$ ryu-manager p3_spr.py
loading app p3_spr.py
loading app ryu.controller.ofp_handler
loading app ryu.topology.switches
loading app ryu.controller.ofp_handler
instantiating app p3_spr.py of ShortestPathSwitch
instantiating app ryu.controller.ofp_handler of OFPHandler
instantiating app ryu.topology.switches of Switches
Link delay from 3 to 2: 0.013410
Link delay from 2 to 3: 0.014211
Link delay from 1 to 4: 0.019088
Link delay from 4 to 1: 0.018980
Link delay from 2 to 1: 0.023208
Link delay from 3 to 4: 0.023696
Link delay from 4 to 3: 0.023681
Link delay from 1 to 2: 0.025692
Link delay from 2 to 3: 0.013548
Link delay from 3 to 2: 0.014273
Link delay from 1 to 4: 0.018533
Link delay from 4 to 1: 0.017914
Link delay from 3 to 4: 0.022388
```

Figure 3: Ryu controller output showing LLDP-based link delay measurements

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
```

Figure 4: pingall results demonstrating full connectivity with shortest path routing

```
(asnvenv) arunangshu@E15:/usr/src/asnvenv/ASN2C$ sudo python3 p3_topo.py
mininet> h4 iperf -s -u &
mininet> h1 iperf -c 10.0.0.4 -u -b 30M
-----
Client connecting to 10.0.0.4, UDP port 5001
Sending 1470 byte datagrams, IPG target: 373.84 us (kalman adjust)
UDP buffer size: 208 KByte (default)
-----
[ 1] local 10.0.0.1 port 50553 connected with 10.0.0.4 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-10.0007 sec 37.5 MBytes 31.5 Mbits/sec
[ 1] Sent 26754 datagrams
```

Figure 5: iperf throughput test results between selected hosts

```
Link delay from 1 to 2: 0.026356
Responding to ARP request for 10.0.0.1 with MAC 4e:fd:4b:1e:52:43
Responding to ARP request for 10.0.0.4 with MAC 2e:0f:1a:2a:db:52
Responding to ARP request for 10.0.0.2 with MAC ae:ed:04:89:bc:5a
Responding to ARP request for 10.0.0.4 with MAC 2e:0f:1a:2a:db:52
Responding to ARP request for 10.0.0.3 with MAC 16:9e:c3:10:c0:74
Responding to ARP request for 10.0.0.4 with MAC 2e:0f:1a:2a:db:52
Link delay from 2 to 3: 0.014055
Link delay from 3 to 2: 0.014803
Link delay from 1 to 4: 0.019134
```

Figure 6: Measured link delays between switches used for path computation

Performance Analysis:

The shortest path routing implementation successfully computes optimal paths based on measured link delays and installs appropriate flow rules. The system demonstrates:

- Efficient path selection avoiding high-delay links
- Reduced latency compared to simple learning switch
- Scalable operation through pre-computed paths
- Automatic adaptation to topology changes via LLDP

The approach provides significant improvements over basic learning switches by considering network conditions and optimizing paths for better performance.

Summary of Part 2

The Shortest Path Routing controller successfully implements a dynamic and efficient routing mechanism that goes beyond basic layer-2 learning. By leveraging LLDP for automatic topology

discovery and real-time delay measurement, the system computes optimal paths using Dijkstra's algorithm and proactively installs bidirectional flows. This approach demonstrates significant improvements in network performance through intelligent path selection based on actual link conditions, providing lower latency and more efficient bandwidth utilization compared to conventional learning switches.

Part 3: Congestion-aware Shortest Path Routing

Implementation Overview

The Congestion-aware Shortest Path Routing controller (`p4_ca_spr.py`) extends the shortest path routing implementation from Part 2 by incorporating dynamic link utilization metrics. The controller monitors network congestion through port statistics and adjusts routing decisions in real-time to avoid congested links. The topology (for which `p3_topo.py` was already given) features a cyclic network with four switches and hosts, with varying link capacities and delays to create realistic congestion scenarios.

Implementation Logic and Approach

Congestion Detection: The controller periodically collects port statistics from all switches, focussing on transmitted packets and dropped packets. The congestion metric is calculated based on the delta of dropped packets between collection intervals, providing a real-time indicator of link saturation.

Dynamic Cost Calculation:

Link costs are computed using the weighted formula given below.

$$\text{Cost} = \text{Delay} + \alpha \times \text{Congestion}$$

where α (`CONGESTION_ALPHA = 0.7`) determines the relative importance of congestion versus delay in path selection.

Flow-level Routing: The implementation operates at the flow level, making routing decisions based on aggregate traffic patterns rather than individual packets. This approach reduces computational overhead while maintaining effective congestion avoidance.

Key Assumptions

1. Packet drops are a reliable indicator of link congestion
2. Congestion patterns are stable enough for periodic path recomputation (10-second intervals)
3. Flow-level routing provides sufficient granularity for congestion management
4. Symmetric link behaviour for bidirectional traffic

Observations of the Experiments, and Analysis

Given below are screenshots from our experiments showing controller output with link delays, pingall output, and iperf output.

```
Updated link costs based on delay and congestion.
Computed shortest paths based on updated link costs.
Installed all computed paths.
Link delay from 2 to 3: 0.012324
Link delay from 3 to 2: 0.012731
Link delay from 4 to 1: 0.016856
Link delay from 1 to 4: 0.019093
Link delay from 4 to 3: 0.021598
Link delay from 1 to 2: 0.024232
Link delay from 2 to 1: 0.023689
Link delay from 3 to 4: 0.023208
Link delay from 2 to 3: 0.013925
Link delay from 3 to 2: 0.014473
Link delay from 4 to 1: 0.017561
```

Figure 7: Controller output showing computed link costs incorporating both delay and congestion

```
(asnvenv) arunangshu@E15:/usr/src/asnvenv/ASN2C$ sudo python3 p3_topo.py
mininet> pingall
*** Ping: testing ping reachability
h1 -> X X X
h2 -> h1 X X
h3 -> h1 h2 X
h4 -> h1 h2 h3
*** Results: 50% dropped (6/12 received)
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
mininet>
```

Figure 8: pingall results demonstrating congestion-aware routing

```
mininet> h1 iperf -c 10.0.0.4 -u -b 10M
-----
Client connecting to 10.0.0.4, UDP port 5001
Sending 1470 byte datagrams, IPG target: 1121.52 us (kalman adjust)
UDP buffer size: 208 KByte (default)
-----
[ 1] local 10.0.0.1 port 58265 connected with 10.0.0.4 port 5001
[ ID] Interval      Transfer    Bandwidth
[ 1] 0.0000-10.0021 sec 12.5 MBytes 10.5 Mbits/sec
[ 1] Sent 8921 datagrams
```

Figure 9: iperf throughput test results showing performance with congestion-aware routing

Performance Analysis:

The congestion-aware routing implementation successfully adapts to network conditions by:

- Dynamically adjusting paths based on real-time congestion measurements
- Avoiding heavily utilised links to prevent packet loss

- Maintaining connectivity while optimizing for both delay and congestion
- Providing better throughput compared to static shortest path routing under congested conditions

The system demonstrates effective load balancing across multiple paths, particularly beneficial in the cyclic topology where alternative routes are available.

Summary of Part 3

The Congestion-aware Shortest Path Routing controller represents a significant advancement over static routing approaches by incorporating real-time network conditions into path computation. By monitoring link utilization and dynamically adjusting routes, the system effectively avoids congestion hotspots and improves overall network performance. The implementation demonstrates practical traffic engineering capabilities that adapt to changing network loads while maintaining efficient packet forwarding.

Conclusion

This assignment successfully implemented and evaluated three progressively sophisticated SDN controller applications, demonstrating the evolution from basic forwarding mechanisms to intelligent, adaptive routing strategies. The Hub Controller provided a foundational understanding of broadcast-based forwarding, while the Learning Switch introduced efficient MAC-based flow installation. The Shortest Path Routing controller advanced this further by incorporating topology awareness and delay-based optimization. Finally, the Congestion-aware Shortest Path Routing controller represented the most sophisticated approach, dynamically adapting to network conditions by combining both delay and congestion metrics for optimal path selection.

The experiments clearly demonstrated that increasing controller intelligence correlates with improved network performance, with the congestion-aware approach providing the most robust solution for real-world network conditions. Each implementation built upon the previous one, showcasing how SDN enables increasingly sophisticated network management through programmable control planes.

List of Files Submitted

Report:

- `report.pdf` (containing all three parts)

Part 1 files:

- `p1_hub.py` (Hub Controller implementation)
- `p1_learning.py` (Learning Switch implementation)

Part 2 Files:

- `p3_spr.py` (Shortest Path Routing implementation)

Part 3 Files:

- `p4_ca_spr.py` (Congestion-aware Shortest Path Routing implementation)

Supporting Files (given along with the assignment):

- `p1_topo.py` (Topology for Part 1)
- `p3_topo.py` (Topology for Parts 2 and 3)